Networking and AWS Architecture Basics

Very basic, beginner-first notes: how networks work, then how AWS builds one

The big picture in 5 lines

- Every device on a network has an **IP address**, like a house number.
- A **port** says which program on that device should receive the data, like a flat number in the house.
- A **subnet** is a group of addresses, like a street. A **router** connects streets.
- A **firewall** decides who is allowed in and out.
- A **VPC** is how AWS gives you your own private network, built from exactly these pieces.

PART 1: NETWORKING BASICS

1.1 IP addresses

An IPv4 address is four numbers from 0 to 255 separated by dots, for example `192.168.1.25`. Each number is 8 bits, so the address is 32 bits and allows about 4.3 billion addresses. That is not enough for the world, so IPv6 (for example `2001:db8::1`) was created, and it is much larger.

Public vs private addresses. Public addresses are reachable on the internet and are unique worldwide. Private addresses are reused by everyone inside their own networks and are not routable on the internet. The three private ranges to memorise:

Range	CIDR	Typical use
`10.0.0.0 - 10.255.255.255`	`10.0.0.0/8`	Large networks, most AWS VPCs
`172.16.0.0 - 172.31.255.255`	`172.16.0.0/12`	Medium networks (the AWS default VPC uses 172.31.0.0/16)
`192.168.0.0 - 192.168.255.255`	`192.168.0.0/16`	Home Wi-Fi routers

1.2 Subnets and CIDR notation

A network is split into smaller **subnets**. **CIDR notation** writes the network as an address plus a number: `10.0.1.0/24`. The number after the slash says how many of the 32 bits are fixed for the network. The rest of the bits are free for devices. **A bigger slash number means a smaller network.**

CIDR	Free bits	Addresses	Example use
`/16`	16	65,536	A whole VPC
`/20`	12	4,096	A large subnet
`/24`	8	256	A typical subnet (10.0.1.0 to 10.0.1.255)
`/28`	4	16	The smallest subnet AWS allows
`/32`	0	1	Exactly one address (used in firewall rules)

Formula: addresses = $2^{(32 - \text{slash})}$. For /24 that is $2^8 = 256$. **AWS keeps 5 addresses in every subnet** (the first four and the last one), so a /24 subnet gives you **251** usable addresses.

Why the CIDR plan matters: two networks with overlapping ranges cannot be connected easily (peering, VPN). Companies plan ranges up front: for example dev uses 10.1.0.0/16, test uses 10.2.0.0/16 and prod uses 10.3.0.0/16.

1.3 Ports and protocols

A server runs many programs at once. The **port** (0 to 65535) selects the program. Rules of the form "allow port 443 from anywhere" are what you write in firewalls and AWS security groups.

Port	Protocol	What it is
22	SSH	Secure remote login to Linux servers
80	HTTP	Web traffic, not encrypted
443	HTTPS	Web traffic encrypted with TLS
53	DNS	Turns names into IP addresses (UDP, sometimes TCP)
3389	RDP	Remote desktop to Windows servers
3306	MySQL	MySQL / Aurora MySQL database
5432	PostgreSQL	PostgreSQL / Aurora PostgreSQL database

TCP vs UDP. TCP checks that every piece of data arrives, in order (web, email, databases, SSH). UDP just sends without checking, which is faster (DNS queries, video calls, streaming). **HTTP** is the language of web requests; **HTTPS** is HTTP wrapped in TLS encryption.

1.4 Key devices and ideas

Term	Meaning in plain words
Router	Connects networks and decides where each packet goes next. Your home Wi-Fi box is one.
Gateway	The exit door of a network: where traffic goes when the destination is outside. The "default gateway" is the router's address in your subnet.
Route table	The list of rules a router follows: "for destination X, send to Y". The rule `0.0.0.0/0` means "everything else".
NAT	Network Address Translation. Lets many private devices share one public address and go out to the internet. Replies come back, but outsiders cannot start a connection in. This is how your phone and laptop share your home internet.
Firewall	Allows or blocks traffic by rules (source, destination, port).
Load balancer	Spreads incoming requests across several servers, and skips unhealthy ones.
DNS	The phonebook that turns `example.com` into an IP address (see the Route 53 notes).
DHCP	Hands out IP addresses automatically when a device joins a network.
VPN	An encrypted tunnel over the internet that joins two networks, or a person to a network.
Bandwidth vs latency	Bandwidth is how much data per second (width of the pipe). Latency is the delay for one message (length of the pipe).

1.5 What happens when you open a website

1. Your browser asks **DNS** for the IP address of the site.
2. Your device sees the address is outside your subnet, so it sends the packet to the **default gateway** (router).
3. Your router uses **NAT** to replace your private address with its public one, and sends the packet onto the internet.
4. The packet crosses many routers to the server's network. A **firewall** checks that port 443 is allowed.
5. A **load balancer** picks one healthy server. A **TCP connection** is set up, then TLS encryption, then the HTTP request and response.

6 The reply travels back the same way, and your router (remembering the NAT mapping) delivers it to your device.

1.6 Practice on your Windows laptop (free)

Open **Command Prompt** and run these. Write down what you see in your own notes: that is great GitHub material.

```
ipconfig                      your IP, subnet mask and default gateway
ipconfig /all                 also shows DNS servers and the MAC address
ping 8.8.8.8                  is the internet reachable? (8.8.8.8 is Google DNS)
ping google.com               tests DNS and connectivity together
nslookup google.com           what IP does this name resolve to?
tracert google.com            every router (hop) between you and the site
netstat -an                   connections and ports your computer has open
curl -I https://example.com   just the HTTP response headers
```

- Find your **default gateway**. Is it a private address? Which of the three private ranges is it in?
- From the subnet mask (255.255.255.0 means /24), how many addresses are in your home network?
- Run `tracert` on two sites. Which has more hops? Does the first hop match your gateway?
- If `ping 8.8.8.8` works but `ping google.com` fails, what is broken? (Answer: DNS).

PART 2: BASIC AWS ARCHITECTURE

2.1 The physical layout

Level	What it is	Why you care
Region	A geographic area (Stockholm = eu-north-1, Mumbai = ap-south-1). Regions are independent.	Choose by user location, law (data residency), price and available services.
Availability Zone (AZ)	One or more separate data centres inside a Region, with their own power and network, linked by fast connections.	Spread your app across at least 2 AZs so one failure does not stop you.
Edge location	Small sites worldwide used by CloudFront (CDN) and Route 53.	Faster delivery to users everywhere.

2.2 VPC: your private network in AWS

A **VPC (Virtual Private Cloud)** is your own isolated network inside one Region. You choose its CIDR range (between /16 and /28 sizes), then divide it into subnets. Nothing outside can reach it unless you build a way in. Every Region already has a **default VPC** (172.31.0.0/16) with one public subnet per AZ, so a first EC2 instance just works. For real projects you create your own.

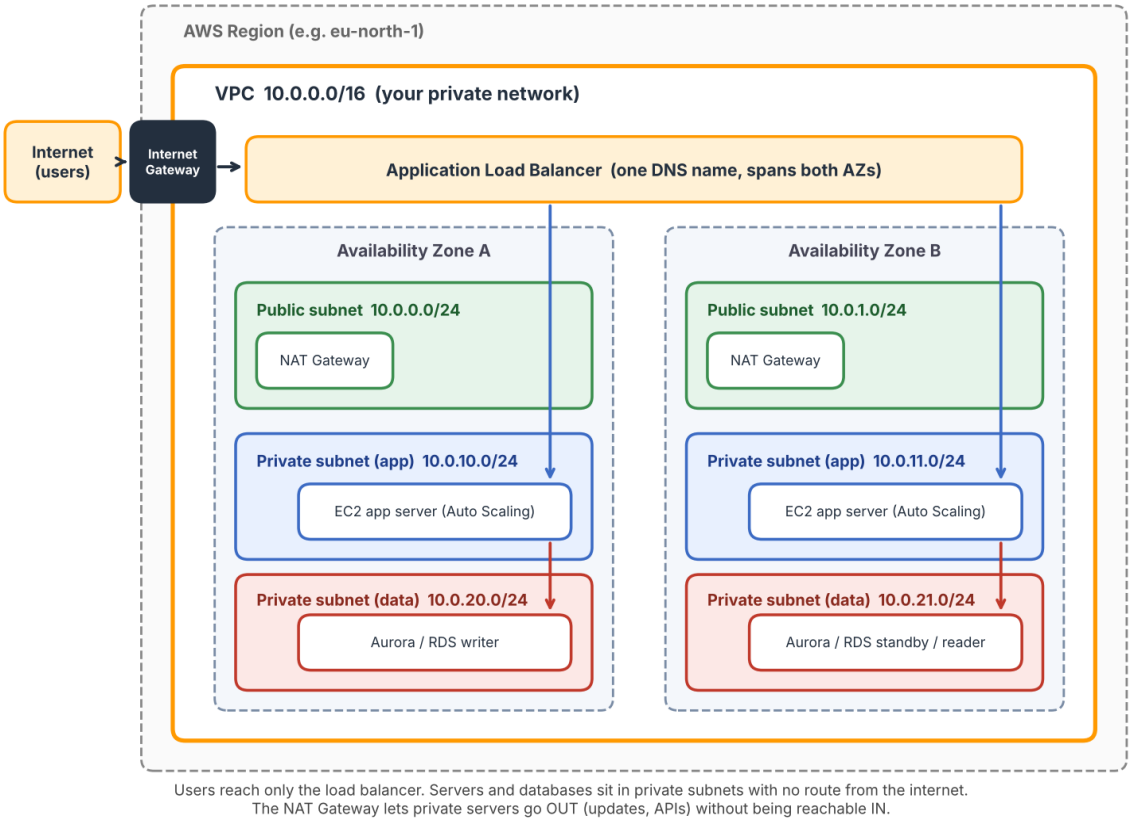


Figure: a standard two-AZ, three-layer design. This is the shape of most real AWS applications.

2.3 The building blocks

Component	What it does	Remember
Subnet	A slice of the VPC's range, living in exactly **one AZ**.	Public or private is decided by its route table, not by its name.
Public subnet	Its route table has `0.0.0.0/0` pointing to an Internet Gateway.	Instances there also need a public IP to be reachable.
Private subnet	No route to an Internet Gateway.	Put app servers and databases here.
Route table	The rules that say where traffic for each destination goes.	Each subnet is linked to one route table.
Internet Gateway (IGW)	The door between the VPC and the internet.	Free. One per VPC. Needs a route to it.
NAT Gateway	Lets private instances go OUT to the internet (patches, APIs) without being reachable IN.	Lives in a **public** subnet. **Costs money per hour plus per GB** and is not Free Tier.
Elastic IP	A fixed public IPv4 address you keep.	Public IPv4 addresses now carry a small hourly charge, so release unused ones.
Security group	A firewall attached to an instance or other resource.	**Stateful**: reply traffic is allowed automatically. Allow rules only. Default: all inbound denied, all outbound allowed.
Network ACL (NACL)	A firewall at the subnet edge.	**Stateless**: you must allow replies yourself. Has allow and deny rules, numbered in order. Defaults are usually fine.
VPC endpoint	A private path from your VPC to an AWS service (S3, DynamoDB and others) without crossing the internet.	The S3 gateway endpoint is free and avoids NAT charges.

Security group vs NACL (a favourite interview question): security groups are stateful, attach to instances and only allow; NACLs are stateless, attach to subnets and can allow or deny. Security groups can also reference *another security group* as the source, for example "allow port 3306 only from the app servers' group".

2.4 How a request flows in the diagram

1. The user types the site name. **Route 53** answers with the address of the **load balancer** (an Alias record).
2. Traffic enters the VPC through the **Internet Gateway** and reaches the **Application Load Balancer**, which sits in the public subnets of both AZs.
3. The load balancer forwards the request to a healthy **EC2 app server** in a **private subnet**. The app servers' security group allows port 80/443 only from the load balancer's security group.
4. The app reads and writes data in **Aurora / RDS** in the data subnets. Its security group allows port 3306 or 5432 only from the app servers' security group.
5. If the app needs to download a patch, the request goes out through the **NAT Gateway**. If one AZ fails, the other keeps serving.

Three layers, three security groups: web tier (open to the internet on 443 through the load balancer), app tier (open only to the load balancer), data tier (open only to the app tier). An attacker who breaks one layer still cannot go straight to the database.

2.5 Connecting networks

Option	Use
VPC peering	Private link between two VPCs. Simple, one-to-one. CIDR ranges must not overlap.
Transit Gateway	A hub that connects many VPCs and on-premises networks, instead of many peerings.
Site-to-Site VPN	Encrypted tunnel over the internet between your office/data centre and a VPC.
Direct Connect	A dedicated physical line to AWS. Stable and fast; slower and more expensive to set up.
PrivateLink	Expose or consume one service privately across VPCs, without opening the whole network.

2.6 Practice labs (VPC itself is free)

Cost note: VPCs, subnets, route tables, internet gateways and security groups are free. **Do not create a NAT Gateway for practice**. A t3.micro instance is within the Free Tier limits for new accounts, but check your own account's Free Tier page. Delete everything at the end.

Lab A: explore the default VPC (5 minutes, no changes)

1. Open **VPC** in the console in **eu-north-1**. Click **Your VPCs** and note the default VPC's CIDR (172.31.0.0/16).
2. Open **Subnets**. Count them: one per AZ. Note each subnet's AZ and CIDR (probably /20, which is 4,096 addresses).
3. Open **Route tables** and select the main one. Look at the **Routes** tab: find the `0.0.0.0/0` route pointing to an `Internet Gateway` (igw-...). That one rule is what makes the subnets public.
4. Open **Security groups** and look at the default group's inbound and outbound rules.

Lab B: build your own small public network by hand

1. **VPC** → **Create VPC** → **VPC only.** Name `lab-vpc`, IPv4 CIDR `10.0.0.0/16`. Create.
2. **Subnets** → **Create subnet**: VPC lab-vpc, name `lab-public-a`, any AZ, CIDR `10.0.1.0/24`. Create a second one: `lab-private-a`, a different AZ, `10.0.11.0/24`.
3. **Internet gateways** → **Create** `lab-igw`, then **Actions** → **Attach to VPC** lab-vpc.
4. **Route tables** → **Create** `lab-public-rt` in lab-vpc. Open it, **Routes** → **Edit** → **Add route**: destination `0.0.0.0/0`, target the Internet Gateway. Then **Subnet associations**: attach **lab-public-a only**.
5. **Subnets** → **lab-public-a** → **Actions** → **Edit subnet settings**: enable **auto-assign public IPv4**.
6. **EC2** → **Launch instance** (Amazon Linux 2023, t3.micro). Under Network settings click Edit: VPC lab-vpc, subnet lab-public-a. Create a security group allowing **SSH 22 from My IP**. Launch, then connect with EC2 Instance Connect. If it connects, your hand-built public subnet works.
7. **Experiment:** remove the 0.0.0.0/0 route from the route table and try to connect again. It fails. Add it back. You have just proven what makes a subnet public.
8. **Clean up:** terminate the instance, then delete the route table, detach and delete the internet gateway, delete both subnets, and delete the VPC.

Troubleshooting "I can't reach my instance" (checklist)

- Is the instance **running** and does it have a **public IP**?
- Does the subnet's **route table** have `0.0.0.0/0` → Internet Gateway, and is the IGW **attached**?
- Does the **security group** allow the port (22/80/443) from **your** current IP? (Home IPs change.)

- Does the **NACL** allow the traffic in *and* the replies out? (Rarely the problem with default NACLs.)
- Is the application **actually listening** on that port? (`sudo ss -tlnp` on the instance).

PART 3: SUMMARY AND SELF-CHECK

Common beginner mistakes

- Opening SSH (22) or a database port to **0.0.0.0/0**. Restrict to your IP or to another security group.
- Putting databases in a **public subnet**.
- Choosing a CIDR that **overlaps** another network you will need to connect later.
- Creating a **NAT Gateway** for a lab and forgetting it (hourly charge).
- Everything in **one AZ**: a single failure takes the whole app down.
- Forgetting that a subnet is only public if its **route table** points to an IGW.

Quick quiz

1. How many addresses are in a `/24` network? How many can you actually use in an AWS subnet of that size?
2. Which of these are private: 10.4.2.1, 172.20.5.9, 172.32.0.1, 192.168.0.7, 8.8.8.8?
3. What single route makes a subnet public?
4. A server in a private subnet must download updates. What do you add, and in which subnet does it live?
5. Security group or NACL: which is stateful, and which one can deny?
6. Why use two AZs?
7. `ping 8.8.8.8` works but `ping google.com` does not. Which service is the likely problem?
8. Which port does HTTPS use? MySQL? SSH?

Answers

1) 256; 251 (AWS reserves 5). 2) 10.4.2.1, 172.20.5.9 and 192.168.0.7. (172.32.0.1 is just outside the private 172.16 to 172.31 range, and 8.8.8.8 is public.) 3) A route `0.0.0.0/0` pointing to an Internet Gateway. 4) A **NAT Gateway**, placed in a **public** subnet, with the private subnet's route table sending `0.0.0.0/0` to it. 5) Security groups are stateful; NACLs can deny. 6) Availability: if one AZ fails the other keeps serving. 7) DNS. 8) 443; 3306; 22.